

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

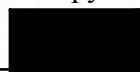
ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Реализация исторических алгоритмов шифрования

Вариант №2

Выполнили:

Жук Анастасия Валерьевна,
студентка группы N3348



(подпись)

Проверил:

Ищенко Алексей Петрович

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
Техническое задание.....	4
Описание продукта, его функционал.....	6
Инструкция по применению.....	8
Код на языке программирования Python.....	10
Примеры работы программы.....	32
ЗАКЛЮЧЕНИЕ.....	41

ВВЕДЕНИЕ

Цель работы – разработка интуитивно понятного настольного приложения с графическим интерфейсом, позволяющего на практике изучить и применять два классических алгоритма шифрования – XOR-шифр (шифрование последовательностей нулей и единиц) и шифр Вижинера (для русского алфавита).

Для достижения поставленной цели необходимо решить следующие задачи:

- реализовать алгоритм XOR-шифра с преобразованием текста в двоичный код (UTF-8) и операцией побитового исключающего ИЛИ;
- реализовать алгоритм шифра Вижинера, адаптированный для работы с русским алфавитом (33 буквы, включая «Ё»);
- создать единый графический интерфейс (GUI) на базе библиотеки tkinter, обеспечивающий удобный выбор алгоритма, ввод данных и отображение результатов;
- обеспечить гибкий ввод и вывод данных: как через текстовые поля интерфейса, так и через работу с файлами;
- добавить вспомогательные функции для удобства пользователя: контекстное меню, горячие клавиши, предустановленные тексты, копирование результатов между полями;
- реализовать надежную обработку ошибок и валидацию входных данных (например, проверка двоичного формата);
- составить подробную инструкцию (справку) внутри программы, объясняющую логику работы с файлами для каждого алгоритма.

ХОД РАБОТЫ

Техническое задание

Написать программу, реализующую два исторических примера алгоритмов шифрования. Продумать интерфейс, руководство пользователя и описание работы алгоритмов (их историей и криптоустойчивостью), для демонстрации алгоритмов пользователю со встроенными примерами текстов (шифруем и дешифруем), а также с возможностью ввода произвольного текста с его шифровкой дешифровкой.

Необходимо реализовать следующие этапы для каждого своего алгоритма:

1. Краткое описание алгоритма шифрования.
2. Определение ограничений на решаемую задачу.
3. Реализация данного алгоритма в виде программного кода.
4. Реализация и описание графического интерфейса.
5. Оформление отчета (включает описание выше и нижеизложенных пунктов со скриншотами работы программы и результатов)

Для защиты программы необходимо предложить несколько (3-4) различных текстов, в которых используется различное количество символов, например: может быть дан текст в котором встречаются много раз только три-четыре буквы, или текст где все буквы сообщения различные, а также вариативность текстов может быть русско-английская.

Алгоритмы шифрования:

1. Шифровка последовательностей нулей и единиц
2. Шифр Вижинера (для русских букв)

Для каждого алгоритма шифрования необходимо написать программу (можно написать одну с меню-выбором из двух заданных алгоритмов), которая работает в двух режимах: шифрования и дешифрования. При любом из режимов программа считывает исходный

текст из окна или файла, шифрует или дешифрует его и записывает или отображает в окно или в другой файл.

Описание продукта, его функционал

Программа представляет собой десктопное приложение, написанное на языке Python с использованием библиотеки tkinter для построения графического интерфейса. Она позволяет пользователям шифровать и дешифровать текстовые сообщения с помощью двух исторически значимых криптографических методов.

Реализованные алгоритмы и принципы их работы:

1. XOR-шифр (шифрование последовательностей нулей и единиц):

— Принцип работы: Основан на операции исключающего ИЛИ (XOR), которая обратима: $(A \text{ XOR } K) \text{ XOR } K = A$. Это свойство позволяет использовать один и тот же ключ для шифрования и дешифрования.

— Реализация в программе: Исходный текст и ключ преобразуются в их двоичное представление в кодировке UTF-8. Затем над полученными битовыми последовательностями выполняется операция XOR. Если ключ короче текста, он циклически повторяется. Результат может отображаться как в виде текста (возможно, с непечатаемыми символами), так и в виде наглядной последовательности нулей и единиц, сгруппированных по 8 бит (байтам). Алгоритм поддерживает работу как с обычным текстом, так и с прямой вставкой двоичного кода для дешифрования.

2. Шифр Вижинера (для русского алфавита):

— Принцип работы: Это полиалфавитный шифр, представляющий собой усложненный шифр Цезаря. Каждая буква открытого текста сдвигается на величину, определяемую соответствующей буквой ключа. Использование ключевого слова делает шифр устойчивее к частотному анализу по сравнению с моноалфавитными шифрами.

— Реализация в программе: Алгоритм адаптирован для 33-букв русского алфавита. Программа автоматически фильтрует введенный текст, оставляя только русские буквы и пробелы (которые не шифруются). Ключ

также приводится к русским буквам и циклически повторяется до длины текста. Шифрование выполняется по формуле $C_i = (P_i + K_i) \bmod 33$, а дешифрование – $P_i = (C_i - K_i) \bmod 33$, где P_i и C_i – позиции букв открытого и зашифрованного текста, а K_i – позиция буквы ключа в алфавите.

Ключевые функциональные возможности интерфейса:

- переключение между двумя алгоритмами с автоматической подстановкой тестового ключа;
- гибкая система ввода: ручной ввод текста, выбор из предустановленных примеров или загрузка данных из файла;
- гибкая система вывода: отображение результата в интерфейсе и/или сохранение в файл (текстовый для Вижинера, двоичный код для XOR-шифра);
- для XOR-шифра предусмотрено отдельное поле для отображения бинарного представления данных;
- встроенная контекстная справка, детально описывающая логику работы с файлами для каждого алгоритма;
- полный набор инструментов для работы с текстом: контекстное меню (Вырезать/Копировать/Вставить/Выделить все/Очистить) и система горячих клавиш (Ctrl+E/D для шифрования/дешифрования, Ctrl+N для справки и др.).

Инструкция по применению

Запуск: Запустите программу, выполнив файл lab5_timp.py.

Выбор алгоритма: В верхней части окна выберите нужный алгоритм шифрования — «Шифровка последовательностей нулей и единиц» (XOR) или «Шифр Вижинера».

Ввод данных:

— **Способ 1 (текст):** В поле «Исходный текст» введите сообщение вручную или выберите пример из выпадающего списка «Примеры текстов».

— **Способ 2 (файл):** Нажмите кнопку «...» рядом с полем «Входной файл», чтобы загрузить текст из файла .txt. Поле для ручного ввода в этом случае можно оставить пустым.

Задание ключа: В поле «Ключ шифрования» введите секретную фразу. Для каждого алгоритма предложен тестовый ключ по умолчанию.

Вывод результата:

— **На экран:** Результат автоматически появится в поле «Результат». Для XOR-шифра двоичный код отобразится в нижнем поле.

— **В файл:** Нажмите «...» у поля «Выходной файл», укажите имя и путь для сохранения. После операции шифрования/дешифрования появится уведомление об успешной записи.

Выполнение операции: Нажмите кнопку «Зашифровать» или «Дешифровать». Для удобства можно использовать горячие клавиши Ctrl+E и Ctrl+D соответственно.

Работа с результатом: Текст из полей «Результат» (для шифра Вижинера) или «Двоичное представление» (для шифровки последовательностей нулей и единиц) можно скопировать в поле ввода для последующих операций с помощью соответствующих кнопок.

Получение помощи: В любой момент нажмите кнопку «Справка» (или Ctrl+H), чтобы открыть подробное руководство по логике работы программы, особенно при использовании файлов.

Код на языке программирования Python

```
import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox, filedialog
import re
import os

# ФУНКЦИИ ДЛЯ ШИФРОВКИ ПОСЛЕДОВАТЕЛЬНОСТЕЙ НУЛЕЙ И ЕДИНИЦ
(XOR ШИФР)

def text_to_binary(text):
    """Преобразование текста в двоичную последовательность (UTF-8)"""
    try:
        # Преобразуем текст в байты (UTF-8)
        bytes_data = text.encode('utf-8')
        # Преобразуем байты в двоичную строку
        binary_str = ".join(format(byte, '08b') for byte in bytes_data)
        return binary_str
    except Exception as e:
        print(f"Ошибка в text_to_binary: {e}")
        return ""

def binary_to_text(binary_str):
    """Преобразование двоичной последовательности в текст (UTF-8)"""
    try:
        # Удаляем все символы, кроме 0 и 1
        binary_str = re.sub(r'[^\01]', "", binary_str)

        # Проверяем, что длина кратна 8
        if len(binary_str) % 8 != 0:
            # Добавляем нули в конец
            padding = 8 - (len(binary_str) % 8)
            binary_str = binary_str + '0' * padding

        # Преобразуем двоичную строку в байты
        bytes_list = []
        for i in range(0, len(binary_str), 8):
            byte_str = binary_str[i:i + 8]
            if len(byte_str) == 8:
                try:
                    byte_val = int(byte_str, 2)
                    bytes_list.append(byte_val)
                except ValueError:
                    continue

        # Преобразуем байты в текст
        if bytes_list:
            try:
```

```

        text = bytes(bytes_list).decode('utf-8', errors='ignore')
        # Удаляем нулевые символы в конце
        text = text.rstrip('\x00')
        return text
    except UnicodeDecodeError:
        return ""
    return ""
except Exception as e:
    print(f"Ошибка в binary_to_text: {e}")
    return ""

def xor_operation(binary1, binary2):
    """Выполнение операции XOR над двумя двоичными строками"""
    # Убедимся, что строки состоят только из 0 и 1
    binary1 = re.sub(r'[^\01]', "", binary1)
    binary2 = re.sub(r'[^\01]', "", binary2)

    # Делаем строки одинаковой длины
    len1 = len(binary1)
    len2 = len(binary2)

    if len1 != len2:
        # Берем минимальную длину
        min_len = min(len1, len2)
        binary1 = binary1[:min_len]
        binary2 = binary2[:min_len]

    # Выполняем XOR
    result = ""
    for i in range(len(binary1)):
        if binary1[i] == binary2[i]:
            result += '0'
        else:
            result += '1'

    return result

def encrypt_binary(text, key):
    """Шифрование текста двоичным методом (XOR шифр)"""
    try:
        # Преобразуем текст в двоичный вид
        text_binary = text_to_binary(text)
        if not text_binary:
            raise ValueError("Не удалось преобразовать текст в двоичный вид")

        # Преобразуем ключ в двоичный вид
        key_binary = text_to_binary(key)
        if not key_binary:

```

```

        raise ValueError("Не удалось преобразовать ключ в двоичный вид")

# Повторяем ключ до длины текста
repeated_key = ""
while len(repeated_key) < len(text_binary):
    repeated_key += key_binary
repeated_key = repeated_key[:len(text_binary)]

# Выполняем XOR
encrypted_binary = xor_operation(text_binary, repeated_key)

# Преобразуем зашифрованную двоичную строку в текст
encrypted_text = binary_to_text(encrypted_binary)

return encrypted_text, encrypted_binary
except Exception as e:
    print(f"Ошибка в encrypt_binary: {e}")
    raise ValueError(f"Ошибка шифрования: {str(e)}")

def decrypt_binary(encrypted_data, key, is_binary_input=False):
    """Дешифрование текста двоичным методом (XOR шифр)"""
    try:
        if is_binary_input:
            # Если ввод в двоичной форме
            encrypted_binary = encrypted_data.replace(' ', '')

            # Проверяем, что это действительно двоичная строка
            if not all(c in '01' for c in encrypted_binary):
                raise ValueError("Введенная строка не является двоичной (должны быть только 0 и 1)")
            else:
                # Если ввод в текстовой форме
                encrypted_binary = text_to_binary(encrypted_data)

        if not encrypted_binary:
            raise ValueError("Не удалось преобразовать данные в двоичный вид")

        # Преобразуем ключ в двоичный вид
        key_binary = text_to_binary(key)
        if not key_binary:
            raise ValueError("Не удалось преобразовать ключ в двоичный вид")

        # Повторяем ключ до длины зашифрованных данных
        repeated_key = ""
        while len(repeated_key) < len(encrypted_binary):
            repeated_key += key_binary
        repeated_key = repeated_key[:len(encrypted_binary)]

        # Выполняем XOR (дешифрование)

```

```

decrypted_binary = xor_operation(encrypted_binary, repeated_key)

# Преобразуем результат в текст
decrypted_text = binary_to_text(decrypted_binary)

return decrypted_text, decrypted_binary
except Exception as e:
    print(f'Ошибка в decrypt_binary: {e}')
    raise ValueError(f'Ошибка дешифрования: {str(e)}')

# ФУНКЦИИ ДЛЯ ШИФРА ВИЖИНЕРА (РУССКИЕ БУКВЫ)

RUSSIAN_ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'

def prepare_vigenere_text(text):
    """Подготовка текста для шифра Вижинера"""
    # Оставляем только русские буквы и пробелы
    text = re.sub(r'[^А-Яа-яЁё\s]', '', text)
    return text.upper()

def prepare_vigenere_key(key, length):
    """Подготовка ключа для шифра Вижинера"""
    # Удаляем все кроме русских букв
    key = re.sub(r'[^А-Яа-яЁё]', '', key)
    key = key.upper()

    if not key:
        key = 'А' # Запасной ключ

    # Повторяем ключ до нужной длины
    repeated_key = ""
    while len(repeated_key) < length:
        repeated_key += key
    return repeated_key[:length]

def encrypt_vigenere(text, key):
    """Шифрование текста шифром Вижинера"""
    prepared_text = prepare_vigenere_text(text)
    prepared_key = prepare_vigenere_key(key, len(prepared_text))

    encrypted_text = ""

    for i, char in enumerate(prepared_text):
        if char == ' ':
            encrypted_text += ' '
        else:

```

```

# Находим индексы букв
text_index = RUSSIAN_ALPHABET.index(char)
key_index = RUSSIAN_ALPHABET.index(prepared_key[i])

# Вычисляем новую позицию
new_index = (text_index + key_index) % len(RUSSIAN_ALPHABET)
encrypted_text += RUSSIAN_ALPHABET[new_index]

return encrypted_text

def decrypt_vigenere(encrypted_text, key):
    """Дешифрование текста шифром Вижинера"""
    prepared_text = prepare_vigenere_text(encrypted_text)
    prepared_key = prepare_vigenere_key(key, len(prepared_text))

    decrypted_text = ""

    for i, char in enumerate(prepared_text):
        if char == ' ':
            decrypted_text += ' '
        else:
            # Находим индексы букв
            text_index = RUSSIAN_ALPHABET.index(char)
            key_index = RUSSIAN_ALPHABET.index(prepared_key[i])

            # Вычисляем исходную позицию
            original_index = (text_index - key_index) % len(RUSSIAN_ALPHABET)
            decrypted_text += RUSSIAN_ALPHABET[original_index]

    return decrypted_text

# ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ ДЛЯ ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

def add_context_menu(widget):
    """Добавляет контекстное меню к виджету"""
    menu = tk.Menu(widget, tearoff=0)
    menu.add_command(label="Вырезать",
                     command=lambda: widget.event_generate("<<Cut>>"))
    menu.add_command(label="Копировать",
                     command=lambda: widget.event_generate("<<Copy>>"))
    menu.add_command(label="Вставить",
                     command=lambda: widget.event_generate("<<Paste>>"))
    menu.add_separator()
    menu.add_command(label="Выделить все",
                     command=lambda: widget.select_range(0, tk.END) if isinstance(widget,
tk.Entry)
                     else widget.tag_add("sel", "1.0", tk.END))
    menu.add_separator()

```

```

menu.add_command(label="Очистить",
                  command=lambda: widget.delete(0, tk.END) if isinstance(widget, tk.Entry)
                  else widget.delete("1.0", tk.END))

# Привязываем правый клик
widget.bind("<Button-3>", lambda e: menu.tk_popup(e.x_root, e.y_root))

return menu

def add_shortcuts_to_text_widget(widget):
    """Добавляет горячие клавиши к текстовому виджету"""
    # Ctrl+C - копирование
    widget.bind("<Control-c>", lambda e: widget.event_generate("<<Copy>>"))
    widget.bind("<Control-C>", lambda e: widget.event_generate("<<Copy>>"))

    # Ctrl+X - вырезание
    widget.bind("<Control-x>", lambda e: widget.event_generate("<<Cut>>"))
    widget.bind("<Control-X>", lambda e: widget.event_generate("<<Cut>>"))

    # Ctrl+V - вставка
    widget.bind("<Control-v>", lambda e: widget.event_generate("<<Paste>>"))
    widget.bind("<Control-V>", lambda e: widget.event_generate("<<Paste>>"))

    # Ctrl+A - выделить все
    widget.bind("<Control-a>", lambda e: widget.tag_add("sel", "1.0", tk.END))
    widget.bind("<Control-A>", lambda e: widget.tag_add("sel", "1.0", tk.END))

    # Ctrl+Z и Ctrl+Y - отмена/повторение
    widget.config(undo=True)
    widget.bind("<Control-z>", lambda e: widget.edit_undo())
    widget.bind("<Control-y>", lambda e: widget.edit_redo())
    widget.bind("<Control-Z>", lambda e: widget.edit_undo())
    widget.bind("<Control-Y>", lambda e: widget.edit_redo())

def add_shortcuts_to_entry_widget(widget):
    """Добавляет горячие клавиши к полю ввода"""
    # Ctrl+C - копирование
    widget.bind("<Control-c>", lambda e: widget.event_generate("<<Copy>>"))
    widget.bind("<Control-C>", lambda e: widget.event_generate("<<Copy>>"))

    # Ctrl+X - вырезание
    widget.bind("<Control-x>", lambda e: widget.event_generate("<<Cut>>"))
    widget.bind("<Control-X>", lambda e: widget.event_generate("<<Cut>>"))

    # Ctrl+V - вставка
    widget.bind("<Control-v>", lambda e: widget.event_generate("<<Paste>>"))
    widget.bind("<Control-V>", lambda e: widget.event_generate("<<Paste>>"))

```

```

# Ctrl+A - выделить все
widget.bind("<Control-a>", lambda e: widget.select_range(0, tk.END))
widget.bind("<Control-A>", lambda e: widget.select_range(0, tk.END))

def create_text_widget_with_shortcuts(parent, **kwargs):
    """Создает текстовое поле с горячими клавишами и меню"""
    text_widget = scrolledtext.ScrolledText(parent, **kwargs)
    add_context_menu(text_widget)
    add_shortcuts_to_text_widget(text_widget)

    # Исправление проблемы с двойной вставкой
    def on_paste(event):
        try:
            # Получаем текст из буфера обмена
            clipboard_text = text_widget.clipboard_get()
            if clipboard_text:
                # Удаляем выделенный текст, если есть
                if text_widget.tag_ranges("sel"):
                    text_widget.delete("sel.first", "sel.last")
                # Вставляем текст
                text_widget.insert("insert", clipboard_text)
            return "break" # Предотвращаем стандартную обработку
        except tk.TclError:
            pass # Если буфер обмена пуст

    # Удаляем старые привязки Ctrl+V и добавляем новую
    text_widget.unbind("<Control-v>")
    text_widget.unbind("<Control-V>")
    text_widget.bind("<Control-v>", on_paste)
    text_widget.bind("<Control-V>", on_paste)

    return text_widget

def create_entry_with_shortcuts(parent, **kwargs):
    """Создает поле ввода с горячими клавишами и меню"""
    entry_widget = ttk.Entry(parent, **kwargs)
    add_context_menu(entry_widget)
    add_shortcuts_to_entry_widget(entry_widget)

    # Исправление проблемы с двойной вставки
    def on_paste(event):
        try:
            # Получаем текст из буфера обмена
            clipboard_text = entry_widget.clipboard_get()
            if clipboard_text:
                # Удаляем выделенный текст, если есть
                if entry_widget.selection_present():
                    entry_widget.delete(tk.SEL_FIRST, tk.SEL_LAST)

```



```

        # Вставляем текст
        entry_widget.insert(tk.INSERT, clipboard_text)
        return "break" # Предотвращаем стандартную обработку
    except tk.TclError:
        pass # Если буфер обмена пуст

# Удаляем старые привязки Ctrl+V и добавляем новую
entry_widget.unbind("<Control-v>")
entry_widget.unbind("<Control-V>")
entry_widget.bind("<Control-v>", on_paste)
entry_widget.bind("<Control-V>", on_paste)

return entry_widget

# ФУНКЦИИ ДЛЯ РАБОТЫ С ФАЙЛАМИ

def read_file(file_path, is_binary=False):
    """Чтение текста из файла"""
    try:
        if is_binary:
            # Для двоичных файлов читаем как есть
            with open(file_path, 'r', encoding='utf-8') as file:
                content = file.read()
            # Удаляем пробелы и переносы строк для двоичных данных
            if all(c in '01 \n\r\t' for c in content):
                content = content.replace(' ', '').replace('\n', '').replace('\r', '').replace('\t', '')
            return content
        else:
            # Для текстовых файлов читаем как текст
            with open(file_path, 'r', encoding='utf-8') as file:
                return file.read()
    except Exception as e:
        raise ValueError(f"Ошибка чтения файла {file_path}: {str(e)}")

def write_to_file(file_path, content):
    """Запись текста в файл"""
    try:
        with open(file_path, 'w', encoding='utf-8') as file:
            file.write(content)
        return True
    except Exception as e:
        raise ValueError(f"Ошибка записи в файл {file_path}: {str(e)}")

def write_binary_to_file(file_path, binary_content, group_by=8):
    """Запись двоичного кода в файл с группировкой"""
    try:
        # Форматируем двоичную строку (группируем по group_by бит)

```

```

if group_by > 0 and binary_content:
    formatted_binary = ''.join([binary_content[i:i + group_by]
                                for i in range(0, len(binary_content), group_by)])
else:
    formatted_binary = binary_content

with open(file_path, 'w', encoding='utf-8') as file:
    file.write(formatted_binary)
return True
except Exception as e:
    raise ValueError(f'Ошибка записи двоичных данных в файл {file_path}: {str(e)}')

```

ГРАФИЧЕСКИЙ ИНТЕРФЕЙС

```

def create_gui():
    """Создание графического интерфейса программы"""

    def on_algorithm_change():
        """Обновление интерфейса при смене алгоритма"""
        algorithm = algorithm_var.get()

        # Обновляем ключ по умолчанию
        if algorithm == "binary":
            key_entry.delete(0, tk.END)
            key_entry.insert(0, default_keys["binary"])
            # Показываем область для двоичного представления
            binary_text.grid()
            binary_mode_frame.grid()
            copy_binary_button.grid(row=0, column=3, padx=3, sticky=tk.EW)
            # Показываем надписи для XOR шифра
            result_label.grid()
            binary_label.grid()
            # Устанавливаем режим ввода по умолчанию
            binary_mode_var.set("text")
        else: # vigenere
            key_entry.delete(0, tk.END)
            key_entry.insert(0, default_keys["vigenere"])
            # Скрываем область для двоичного представления
            binary_text.grid_remove()
            binary_mode_frame.grid_remove()
            copy_binary_button.grid_forget()
            # Скрываем надписи для XOR шифра
            result_label.grid_remove()
            binary_label.grid_remove()

        # Очищаем результаты
        output_text.delete("1.0", tk.END)
        binary_text.delete("1.0", tk.END)

```

```

def on_test_text_select(event=None):
    """Загрузка выбранного тестового текста"""
    text_name = text_var.get()
    input_text.delete("1.0", tk.END)
    input_text.insert("1.0", test_texts[text_name])

    # Очищаем результаты
    output_text.delete("1.0", tk.END)
    binary_text.delete("1.0", tk.END)

    # Если выбран двоичный шифр, устанавливаем режим по умолчанию
    if algorithm_var.get() == "binary":
        binary_mode_var.set("text")

def select_input_file():
    """Выбор входного файла"""
    filename = filedialog.askopenfilename(
        title="Выберите входной файл",
        filetypes=[
            ("Текстовые файлы", "*.txt"),
            ("Все файлы", "*.*")
        ]
    )
    if filename:
        input_file_var.set(filename)

def select_output_file():
    """Выбор выходного файла"""
    filename = filedialog.asksaveasfilename(
        title="Выберите выходной файл",
        defaultextension=".txt",
        filetypes=[
            ("Текстовые файлы", "*.txt"),
            ("Все файлы", "*.*")
        ]
    )
    if filename:
        output_file_var.set(filename)

def clear_file_selections():
    """Очистка выбора файлов"""
    input_file_var.set("")
    output_file_var.set("")

def process_with_files(input_file, output_file, operation_func, is_decrypt=False):
    """Обработка с учетом файлов"""
    try:
        # 1. Получаем входные данные
        input_data = ""
        if input_file and os.path.exists(input_file):

```

```

# Читаем из файла
if algorithm_var.get() == "binary" and is_decrypt:
    # Для дешифрования двоичного шифра определяем, является ли файл
двоичным
    try:
        # Пробуем прочитать как двоичный файл
        with open(input_file, 'r', encoding='utf-8') as f:
            content = f.read()
            # Проверяем, содержит ли файл только 0, 1 и пробелы
            if all(c in '01 \n\r\t' for c in content):
                # Это двоичный файл
                input_data = content.replace(' ', '').replace('\n', '').replace('\r', '').replace('\t',
                                                                                               "")
                # Автоматически устанавливаем режим "двоичный код"
                binary_mode_var.set("binary")
            else:
                # Это текстовый файл
                input_data = content
                binary_mode_var.set("text")
    except:
        # Если не удалось прочитать как текст, пробуем как двоичные данные
        with open(input_file, 'rb') as f:
            binary_data = f.read()
            input_data = ''.join(format(byte, '08b') for byte in binary_data)
            binary_mode_var.set("binary")
    else:
        # Для шифрования или для Вижинера читаем как текст
        input_data = read_file(input_file)
    else:
        # Читаем из текстового поля
        input_data = input_text.get("1.0", tk.END).strip()
        if not input_data:
            raise ValueError("Введите текст или выберите входной файл")

# 2. Получаем ключ
key_data = key_entry.get().strip()
if not key_data:
    raise ValueError("Введите ключ шифрования")

# 3. Выполняем операцию (шифрование или дешифрование)
if algorithm_var.get() == "binary" and is_decrypt:
    # Для дешифрования XOR шифра передаем дополнительный параметр
    is_binary_input = binary_mode_var.get() == "binary"
    result_text, result_binary = operation_func(input_data, key_data, is_binary_input)
else:
    # Для шифрования или для Вижинера
    result_text, result_binary = operation_func(input_data, key_data)

# 4. Обрабатываем выходные данные
output_displayed = False

```

```

if output_file:
    # Записываем в файл
    if algorithm_var.get() == "binary":
        # Для XOR шифра записываем двоичный код
        write_binary_to_file(output_file, result_binary, group_by=8)
        file_content_type = "двоичный код"
    else:
        # Для Вижинера записываем текст
        write_to_file(output_file, result_text)
        file_content_type = "текст"

    messagebox.showinfo("Успех",
                        f"Результат сохранен в файл:\n{output_file}\n"
                        f"Тип содержимого: {file_content_type}")

# 5. Отображаем результат в интерфейсе (если не записали в файл или если нет
выходного файла)
if not output_file or input_file: # Показываем если нет выходного файла ИЛИ есть
входной файл
    output_text.delete("1.0", tk.END)
    output_text.insert("1.0", result_text)

    if algorithm_var.get() == "binary" and result_binary:
        binary_text.delete("1.0", tk.END)
        formatted_binary = ''.join([result_binary[i:i + 8]
                                     for i in range(0, len(result_binary), 8)])
        binary_text.insert("1.0", formatted_binary)

    output_displayed = True

return True

except Exception as e:
    error_msg = str(e)
    messagebox.showerror("Ошибка", f"Произошла ошибка:\n{error_msg}")
    return False

def encrypt_vigenere_wrapper(text, key):
    """Обертка для шифрования Вижинера для совместимости с process_with_files"""
    result_text = encrypt_vigenere(text, key)
    return result_text, "" # Возвращаем пустую строку для двоичного представления

def decrypt_vigenere_wrapper(text, key):
    """Обертка для дешифрования Вижинера для совместимости с
process_with_files"""
    result_text = decrypt_vigenere(text, key)
    return result_text, "" # Возвращаем пустую строку для двоичного представления

def encrypt_text():

```

```

"""Шифрование текста"""
try:
    input_file = input_file_var.get()
    output_file = output_file_var.get()

    algorithm = algorithm_var.get()

    if algorithm == "binary":
        process_with_files(input_file, output_file, encrypt_binary, is_decrypt=False)
    else: # vigenere
        process_with_files(input_file, output_file, encrypt_vigenere_wrapper,
is_decrypt=False)

except Exception as e:
    error_msg = str(e)
    messagebox.showerror("Ошибка", f"Произошла ошибка при
шифровании:\n{error_msg}")

def decrypt_text():
    """Дешифрование текста"""
    try:
        input_file = input_file_var.get()
        output_file = output_file_var.get()

        algorithm = algorithm_var.get()

        if algorithm == "binary":
            # Создаем обертку для decrypt_binary
            def decrypt_binary_wrapper(text, key, is_binary_input=False):
                return decrypt_binary(text, key, is_binary_input)

            process_with_files(input_file, output_file, decrypt_binary_wrapper,
is_decrypt=True)
        else: # vigenere
            process_with_files(input_file, output_file, decrypt_vigenere_wrapper,
is_decrypt=False)

    except Exception as e:
        error_msg = str(e)
        messagebox.showerror("Ошибка", f"Произошла ошибка при
дешифровании:\n{error_msg}")

def clear_texts():
    """Очистка всех текстовых полей"""
    input_text.delete("1.0", tk.END)
    output_text.delete("1.0", tk.END)
    binary_text.delete("1.0", tk.END)

def show_help():
    """Показ справки по программе"""

```

```
algorithm = algorithm_var.get()

if algorithm == "binary":
    help_text = ""
    ШИФРОВКА ПОСЛЕДОВАТЕЛЬНОСТЕЙ НУЛЕЙ И ЕДИНИЦ С
ПОДДЕРЖКОЙ ФАЙЛОВ
```

РАБОТА С ФАЙЛАМИ:

1. БЕЗ ФАЙЛОВ (стандартный режим):
 - Вводите текст в поле "Исходный текст"
 - Результат отображается в окне
2. ТОЛЬКО ВХОДНОЙ ФАЙЛ:
 - Выберите файл для чтения текста
 - Результат отображается в окне
3. ТОЛЬКО ВЫХОДНОЙ ФАЙЛ:
 - Введите текст в поле "Исходный текст"
 - Выберите файл для сохранения результата
 - Результат сохраняется в файл (двоичный код)
 - Появляется сообщение об успешном сохранении
4. ОБА ФАЙЛА (входной и выходной):
 - Выберите файл для чтения текста
 - Выберите файл для сохранения результата
 - Результат сохраняется в файл
 - Появляется сообщение об успешном сохранении
 - Результат также отображается в окне

КАК ВЫБРАТЬ ФАЙЛ:

- Нажмите кнопку "..." рядом с полем "Входной файл" или "Выходной файл"
- Выберите файл в диалоговом окне
- Для очистки выбора нажмите кнопку "Очистить файлы"

ПРИМЕЧАНИЯ:

- Для шифровки последовательностей нулей и единиц в файл сохраняется двоичный код (0 и 1)
- Для шифра Вижинера в файл сохраняется текст
- Поддерживается кодировка UTF-8
- Файлы можно выбирать в любом порядке

```
else: # vigenere
    help_text = ""
    ШИФР ВИЖИНЕРА (ДЛЯ РУССКИХ БУКВ) С ПОДДЕРЖКОЙ ФАЙЛОВ
```

РАБОТА С ФАЙЛАМИ:

1. БЕЗ ФАЙЛОВ (стандартный режим):
 - Вводите текст в поле "Исходный текст"

- Результат отображается в окне
2. ТОЛЬКО ВХОДНОЙ ФАЙЛ:
- Выберите файл для чтения текста
 - Результат отображается в окне
3. ТОЛЬКО ВЫХОДНОЙ ФАЙЛ:
- Введите текст в поле "Исходный текст"
 - Выберите файл для сохранения результата
 - Результат сохраняется в файл (текст)
 - Появляется сообщение об успешном сохранении
4. ОБА ФАЙЛА (входной и выходной):
- Выберите файл для чтения текста
 - Выберите файл для сохранения результата
 - Результат сохраняется в файл
 - Появляется сообщение об успешном сохранении
 - Результат также отображается в окне

КАК ВЫБРАТЬ ФАЙЛ:

- Нажмите кнопку "..." рядом с полем "Входной файл" или "Выходной файл"
- Выберите файл в диалоговом окне
- Для очистки выбора нажмите кнопку "Очистить файлы"

ПРИМЕЧАНИЯ:

- Для шифра Вижинера в файл сохраняется текст
- Поддерживается кодировка UTF-8
- Файлы можно выбирать в любом порядке
- Из файла читается только русский текст (остальные символы игнорируются)

```
# Создаем окно справки
help_window = tk.Toplevel(root)
help_window.title("Справка по программе (работа с файлами)")
help_window.geometry("800x600")
help_window.resizable(True, True)

# Делаем окно модальным
help_window.transient(root)
help_window.grab_set()

# Создаем контейнер для справки
help_container = ttk.Frame(help_window)
help_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

help_text_area = scrolledtext.ScrolledText(help_container, wrap=tk.WORD,
                                             font=("Arial", 10), bg="#ffffff")
help_text_area.pack(fill=tk.BOTH, expand=True)
help_text_area.insert("1.0", help_text)
help_text_area.config(state=tk.DISABLED)
```



```

# Кнопка закрытия
tk.Button(help_window, text="Закрыть",
command=help_window.destroy).pack(pady=10)

# Привязываем клавишу Esc к закрытию окна
help_window.bind("<Escape>", lambda e: help_window.destroy())

def copy_binary_to_input():
    """Копирует двоичный код в поле ввода"""
    binary_data = binary_text.get("1.0", tk.END).strip()
    if binary_data:
        input_text.delete("1.0", tk.END)
        input_text.insert("1.0", binary_data)
        # Автоматически выбираем режим "Двоичный код"
        binary_mode_var.set("binary")

def copy_result_to_input():
    """Копирует результат в поле ввода"""
    result_data = output_text.get("1.0", tk.END).strip()
    if result_data:
        input_text.delete("1.0", tk.END)
        input_text.insert("1.0", result_data)

# Если выбран двоичный шифр, устанавливаем режим "Текст" для
дешифрования
if algorithm_var.get() == "binary":
    binary_mode_var.set("text")

def add_global_shortcuts():
    """Добавляет глобальные горячие клавиши"""
    # Ctrl+E - шифровать
    root.bind("<Control-e>", lambda e: encrypt_text())
    root.bind("<Control-E>", lambda e: encrypt_text())

    # Ctrl+D - дешифровать
    root.bind("<Control-d>", lambda e: decrypt_text())
    root.bind("<Control-D>", lambda e: decrypt_text())

    # Ctrl+L - очистить
    root.bind("<Control-l>", lambda e: clear_texts())
    root.bind("<Control-L>", lambda e: clear_texts())

    # Ctrl+H - справка
    root.bind("<Control-h>", lambda e: show_help())
    root.bind("<Control-H>", lambda e: show_help())

    # Ctrl+Q - выход
    root.bind("<Control-q>", lambda e: root.quit())
    root.bind("<Control-Q>", lambda e: root.quit())

```

```

# Ctrl+R - копировать результат в поле ввода
root.bind("<Control-r>", lambda e: copy_result_to_input())
root.bind("<Control-R>", lambda e: copy_result_to_input())

# Ctrl+I - выбрать входной файл
root.bind("<Control-i>", lambda e: select_input_file())
root.bind("<Control-I>", lambda e: select_input_file())

# Ctrl+O - выбрать выходной файл
root.bind("<Control-o>", lambda e: select_output_file())
root.bind("<Control-O>", lambda e: select_output_file())

# =====
# ОСНОВНОЕ ОКНО ПРОГРАММЫ
# =====

# Создаем главное окно
root = tk.Tk()
root.title("Исторические алгоритмы шифрования")
root.geometry("1200x950")
root.minsize(1000, 800)

# Настройка стилей
style = ttk.Style()
style.theme_use('clam')
root.configure(bg="#f0f0f0")

# Данные для программы (убраны лишние тексты)
test_texts = {
    "Короткий русский": "ПРИВЕТ МИР",
    "Длинный русский": "Съешь же ещё этих мягких французских булок, да выпей чаю. Тридцать три богатыря, в чешуе златой, по очереди выходят из моря и дозором обходят дозором владения дядьки Черномора.",
    "Английский текст": "The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs.",
    "Текст с повторениями": "ААА БББ ВВВ ГГГ ДДД ЕЕЕ ЁЁЁ ЖЖЖ ЗЗЗ ИИИ ЙЙЙ ККК ЛЛЛ МММ ННН ООО ППП"
}

default_keys = {
    "binary": "SECRET_KEY_2024",
    "vigenere": "ПАРОЛЬ"
}

# Переменные для файлов
input_file_var = tk.StringVar()
output_file_var = tk.StringVar()

# Основной контейнер

```

```

main_frame = ttk.Frame(root, padding="10")
main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

# Настройка расширяемости
root.columnconfigure(0, weight=1)
root.rowconfigure(0, weight=1)

# Настройка веса строк и колонок в main_frame
for i in range(18): # Всего 18 строк
    main_frame.rowconfigure(i, weight=0)

# Даем вес только строкам с текстовыми полями
main_frame.rowconfigure(10, weight=1) # Исходный текст
main_frame.rowconfigure(13, weight=1) # Результат
main_frame.rowconfigure(15, weight=1) # Двоичное представление

main_frame.columnconfigure(0, weight=0)
main_frame.columnconfigure(1, weight=1)
main_frame.columnconfigure(2, weight=0)

# Заголовок
title_label = ttk.Label(main_frame,
                        text="Исторические алгоритмы шифрования",
                        font=("Arial", 16, "bold"))
title_label.grid(row=0, column=0, columnspan=3, pady=(0, 10))

# Выбор алгоритма
ttk.Label(main_frame, text="Выберите алгоритм шифрования:",
          font=("Arial", 11)).grid(row=1, column=0, sticky=tk.W, pady=5)

algorithm_var = tk.StringVar(value="binary")
algorithms_frame = ttk.Frame(main_frame)
algorithms_frame.grid(row=1, column=1, sticky=tk.W, pady=5)

ttk.Radiobutton(algorithms_frame, text="Шифровка последовательностей нулей и
единиц",
                variable=algorithm_var, value="binary",
                command=on_algorithm_change).pack(side=tk.LEFT, padx=5)
ttk.Radiobutton(algorithms_frame, text="Шифр Вижинера (для русских букв)",
                variable=algorithm_var, value="vigenere",
                command=on_algorithm_change).pack(side=tk.LEFT, padx=5)

# Режим ввода для двоичного шифра
binary_mode_var = tk.StringVar(value="text")
binary_mode_frame = ttk.Frame(main_frame)
binary_mode_frame.grid(row=2, column=0, columnspan=2, pady=5, sticky=tk.W)
binary_mode_frame.grid_remove() # Скрываем по умолчанию

ttk.Label(binary_mode_frame, text="Режим ввода для дешифрования:",
          font=("Arial", 11)).pack(side=tk.LEFT, padx=5)

```

```

ttk.Radiobutton(binary_mode_frame, text="Текст",
                variable=binary_mode_var, value="text").pack(side=tk.LEFT, padx=5)
ttk.Radiobutton(binary_mode_frame, text="Двоичный код (0 и 1)",
                variable=binary_mode_var, value="binary").pack(side=tk.LEFT, padx=5)

# Ключ шифрования
ttk.Label(main_frame, text="Ключ шифрования:",
          font=("Arial", 11)).grid(row=3, column=0, sticky=tk.W, pady=5)
key_entry = create_entry_with_shortcuts(main_frame, width=50)
key_entry.grid(row=3, column=1, sticky=(tk.W, tk.E), pady=5, padx=(0, 10))
key_entry.insert(0, default_keys["binary"])

# Входной файл
ttk.Label(main_frame, text="Входной файл (опционально):",
          font=("Arial", 11)).grid(row=4, column=0, sticky=tk.W, pady=5)

input_file_frame = ttk.Frame(main_frame)
input_file_frame.grid(row=4, column=1, sticky=(tk.W, tk.E), pady=5, padx=(0, 10))

input_file_entry = ttk.Entry(input_file_frame, textvariable=input_file_var, width=40)
input_file_entry.pack(side=tk.LEFT, fill=tk.X, expand=True)
add_context_menu(input_file_entry)

ttk.Button(input_file_frame, text="...", width=3,
          command=select_input_file).pack(side=tk.LEFT, padx=(5, 0))

# Выходной файл
ttk.Label(main_frame, text="Выходной файл (опционально):",
          font=("Arial", 11)).grid(row=5, column=0, sticky=tk.W, pady=5)

output_file_frame = ttk.Frame(main_frame)
output_file_frame.grid(row=5, column=1, sticky=(tk.W, tk.E), pady=5, padx=(0, 10))

output_file_entry = ttk.Entry(output_file_frame, textvariable=output_file_var, width=40)
output_file_entry.pack(side=tk.LEFT, fill=tk.X, expand=True)
add_context_menu(output_file_entry)

ttk.Button(output_file_frame, text="...", width=3,
          command=select_output_file).pack(side=tk.LEFT, padx=(5, 0))

# Кнопка очистки файлов
file_buttons_frame = ttk.Frame(main_frame)
file_buttons_frame.grid(row=6, column=0, columnspan=3, pady=10, sticky=tk.W)

ttk.Button(file_buttons_frame, text="Очистить выбор файлов",
          command=clear_file_selections, width=25).pack(side=tk.LEFT, padx=(0, 10))

# Разделитель
separator = ttk.Separator(main_frame, orient='horizontal')
separator.grid(row=7, column=0, columnspan=3, sticky=(tk.W, tk.E), pady=10)

```

```

# Выбор тестового текста
tk.Label(main_frame, text="Примеры текстов:",
         font=("Arial", 11)).grid(row=8, column=0, sticky=tk.W, pady=5)

text_var = tk.StringVar(value="Короткий русский")
text_combobox = ttk.Combobox(main_frame, textvariable=text_var,
                              values=list(test_texts.keys()),
                              state="readonly", width=40)
text_combobox.grid(row=8, column=1, sticky=tk.W, pady=5, padx=(0, 10))
text_combobox.bind("<<ComboboxSelected>>", on_test_text_select)

# Область для ввода исходного текста
tk.Label(main_frame, text="Исходный текст (если нет входного файла):",
         font=("Arial", 11)).grid(row=9, column=0, sticky=tk.NW, pady=5)

input_text = create_text_widget_with_shortcuts(main_frame, width=60, height=8,
                                                wrap=tk.WORD, font=("Courier", 10),
                                                bg="#ffffff")
input_text.grid(row=10, column=0, columnspan=3, sticky=(tk.W, tk.E, tk.N, tk.S),
               pady=5)
input_text.insert("1.0", test_texts["Короткий русский"])

# Объединенные кнопки управления (одна строка)
button_frame = ttk.Frame(main_frame)
button_frame.grid(row=11, column=0, columnspan=3, pady=15, sticky=tk.EW)

# Настройка колонок для равномерного распределения кнопок
for i in range(7):
    button_frame.columnconfigure(i, weight=1)

# Кнопки шифрования/дешифрования
tk.Button(button_frame, text="Зашифровать",
         command=encrypt_text).grid(row=0, column=0, padx=3, sticky=tk.EW)
tk.Button(button_frame, text="Дешифровать",
         command=decrypt_text).grid(row=0, column=1, padx=3, sticky=tk.EW)

# Кнопки копирования
copy_result_button = tk.Button(button_frame, text="Копировать результат",
                               command=copy_result_to_input)
copy_result_button.grid(row=0, column=2, padx=3, sticky=tk.EW)

copy_binary_button = tk.Button(button_frame, text="Копировать двоичный код",
                               command=copy_binary_to_input)
copy_binary_button.grid(row=0, column=3, padx=3, sticky=tk.EW)

# Кнопки очистки и справки
tk.Button(button_frame, text="Очистить поля",
         command=clear_texts).grid(row=0, column=4, padx=3, sticky=tk.EW)
tk.Button(button_frame, text="Справка",

```

```

        command=show_help).grid(row=0, column=5, padx=3, sticky=tk.EW)

# Кнопка выхода
tk.Button(button_frame, text="Выход",
        command=root.quit).grid(row=0, column=6, padx=3, sticky=tk.EW)

# Область для вывода результата (надпись и поле)
result_label = tk.Label(main_frame, text="Результат:",
        font=("Arial", 11))
result_label.grid(row=12, column=0, columnspan=3, sticky=tk.NW, pady=5)

output_text = create_text_widget_with_shortcuts(main_frame, width=60, height=8,
        wrap=tk.WORD, font=("Courier", 10),
        bg="#ffffff")
output_text.grid(row=13, column=0, columnspan=3, sticky=(tk.W, tk.E, tk.N, tk.S),
        pady=5)

# Область для двоичного представления (надпись и поле)
binary_label = tk.Label(main_frame, text="Двоичное представление:",
        font=("Arial", 11))
binary_label.grid(row=14, column=0, columnspan=3, sticky=tk.NW, pady=5)

binary_text = create_text_widget_with_shortcuts(main_frame, width=60, height=6,
        wrap=tk.WORD, font=("Courier", 9),
        bg="#ffffff")
binary_text.grid(row=15, column=0, columnspan=3, sticky=(tk.W, tk.E, tk.N, tk.S),
        pady=5)

# Добавляем глобальные горячие клавиши
add_global_shortcuts()

# Инициализация интерфейса
on_algorithm_change()

# Центрируем окно на экране
root.update_idletasks()
width = root.winfo_width()
height = root.winfo_height()
x = (root.winfo_screenwidth() // 2) - (width // 2)
y = (root.winfo_screenheight() // 2) - (height // 2)
root.geometry(f'{width}x{height}+{x}+{y}')

# Запуск основного цикла
root.mainloop()

# ЗАПУСК ПРОГРАММЫ

if __name__ == "__main__":
    # Запуск графического интерфейса

```

`create_gui()`

Примеры работы программы

Исторические алгоритмы шифрования

Выберите алгоритм шифрования: ☒ Шифровка последовательностей нулей и единиц ☐ Шифр Вижнера (для русских букв)

Режим ввода для дешифрования: ☒ Текст ☐ Двоичный код (0 и 1)

Ключ шифрования: SECRET_KEY_2024

Входной файл (опционально):

Выходной файл (опционально):

Очистить выбор файлов

Примеры текстов: Короткий русский

Исходный текст (если нет входного файла):

ПРИВЕТ МИР

Зашифровать Дешифровать Копировать результат Копировать двоичный код Очистить поля Справка Выход

Результат:

Двоичное представление:

Рисунок 1 – Графический интерфейс для шифровки последовательностей нулей и единиц

Примеры текстов: Короткий русский

Исходный текст (если нет входного файла):

ПРИВЕТ МИР

Зашифровать Дешифровать Копировать результат Копировать двоичный код Очистить поля Справка Выход

Результат:

~ ~ ~ ~ ~

Двоичное представление:

10000011 11011010 10010011 11110010 10010101 11001100 10001111 11011001 10010101 11001100 10001111 10010000 00010000 11100010 10101000 10000011 11011101 10010011 11110010

Рисунок 2 – Шифровка последовательностей нулей и единиц. Короткий русский текст

Примеры текстов: Английский текст

Исходный текст (если нет входного файла):

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs.

Зашифровать Дешифровать Копировать результат Копировать двоичный код Очистить поля Справка Выход

Результат:

0-4z4!6(.y=@_Ez#,"e">450)FWFz1+7e8>1<y;)WfG 9e94k'6'fG{8:re%;3l/'#:\f"}0, e>*,6w

Двоичное представление:

00000111 00101101 00100110 01110010 00110100 00100001 00110110 00101000 00101110 01111001 00111101 01000000 01011111 01000101 01011010 01110011 00100011 00101000 00101010 01100101 00111110 00101010 00100110
00110101 00101010 01111111 01011101 01000110 01010011 01000110 01110011 00110001 00101011 01101111 01100101 00111000 00111110 00110001 00111001 00111011 01011101 01010111 00010100 00010100 00000011
00100100 00100000 00110001 01100101 00111001 00100110 01101011 00101111 00110110 00100111 00010010 01000111 01011011 01000000 00111011 01100101 00100101 01110111 00110011 01111111 00111111 00101010
00100011 01111010 01011100 00010000 01111110 01011011 00100010 00110000 00101100 00100000 0100101 00111110 00101010 00101100 00101010 01101111

Рисунок 6 – Шифровка последовательностей нулей и единиц. Английский текст

Примеры текстов: Английский текст

Исходный текст (если нет входного файла):

00000111 00101101 00100110 01110010 00110100 00100001 00110110 00101000 00101110 01111001 00111101 01000000 01011111 01000101 01011010 01110011 00100011 00101000 00101010 01100101 00111110 00101010 01100101
00111110 00101010 00100110 00110101 00101010 01111111 01011101 00100110 01010111 01000110 01110011 00110001 00101011 00110111 0100101 00111000 00111110 00110001 00111100 01111001 00111001
00111011 01011101 01010111 00011100 00101010 00000011 00100100 00100000 00111001 01100101 00111001 00100110 01101011 00100111 00110110 00100111 00010010 01000111 01010111 01000000
00110011 01000101 00100101 00111011 00110001 00110001 01111111 00101111 00101010 00100011 00111010 01011100 00100000 01011110 01011101 00100010 00110000 00101100 00100000 01100101
01111110 00101010 00101100 00101010 01110111

Зашифровать Дешифровать Копировать результат Копировать двоичный код Очистить поля Справка Выход

Результат:

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs.

Двоичное представление:

01010100 01101000 01100101 00100000 01110001 01101011 01101001 01100011 01101011 00100000 01100010 01110010 01101111 01101110 00100000 01001110 01101111 01111000 00100000 01101010 01110101 01101101
01110000 01110011 00100000 01101111 01101110 01100101 01110010 00100000 01110100 01100101 00100000 01101100 01100001 01111010 01111001 00100000 01100100 01101111 00110111 00101110 00100000 01010000
01100001 01100011 01101011 00100000 01101101 01111001 00100000 01100010 01101111 01111000 00100000 01110111 01101001 01101100 01101000 00100000 01001110 01101001 01101110 01100101 00100000 01100100 01101111
01110101 01100101 01111110 00100000 00110100 01101001 01111001 01101011 01101111 01100010 00100000 01101010 01110101 01100111 01100111 00110011 00101110

Рисунок 7 – Дешифровка последовательностей нулей и единиц. Английский текст

Примеры текстов: Текст с повторениями

Исходный текст (если нет входного файла):

aaa bbb bbb gtt ddd eee eee xxx zzz нии ййй ккк ллл ммм ннн ооо ппп

Зашифровать Дешифровать Копировать результат Копировать двоичный код Очистить поля Справка Выход

Результат:

4b0t20' 4e4b0t20'

Двоичное представление:

10000011 11010101 10010011 11000010 10010101 11000100 01111111 10011011 11010100 10001001 11001110 11100010 10100001 00010010 11100100 11000001 10010101 11010001 10000010 11010111 01110100 10001111 11011000
10010101 11001010 10001111 10100001 00010000 11100010 10100000 10000011 11010001 10010011 10000110 10001010 10010011 11010000 10001001 11001010 00010010 11100000 10110011 11100100 11010010
10010101 11000010 01110010 10010101 11000010 00011111 11011101 10010101 11001111 01111111 11100010 10100111 11100010 10100011 10000011 11010010 01100011 10000010 11011101 10000100 10000111 10011011 01111101
01111001 10001111 10101011 11100000 10101011 11100100 11001011 01100101 10010011 10010101 11001110 10001111 11010001 01100101 10001001 11000100 11100010 10101011 11100010 10101111 01110011 10010101

Рисунок 8 – Шифровка последовательностей нулей и единиц. Текст с повторениями

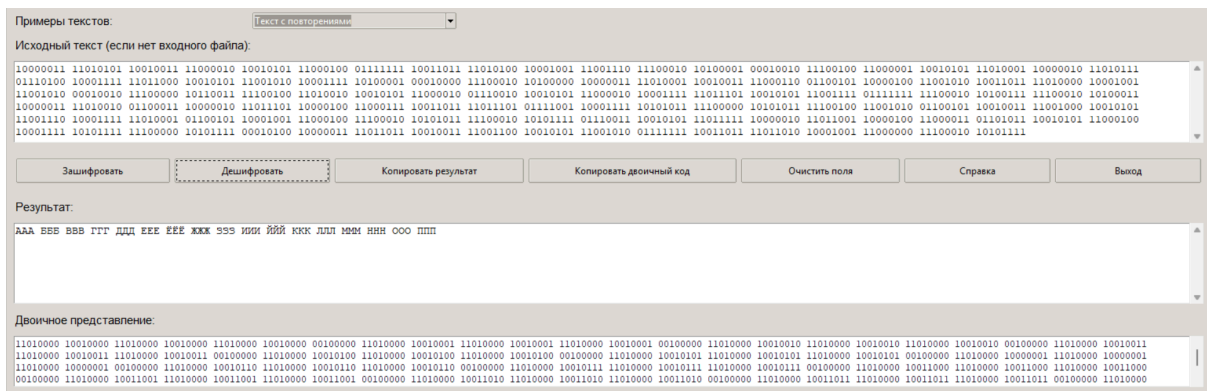


Рисунок 9 – Дешифровка последовательностей нулей и единиц. Текст с повторениями

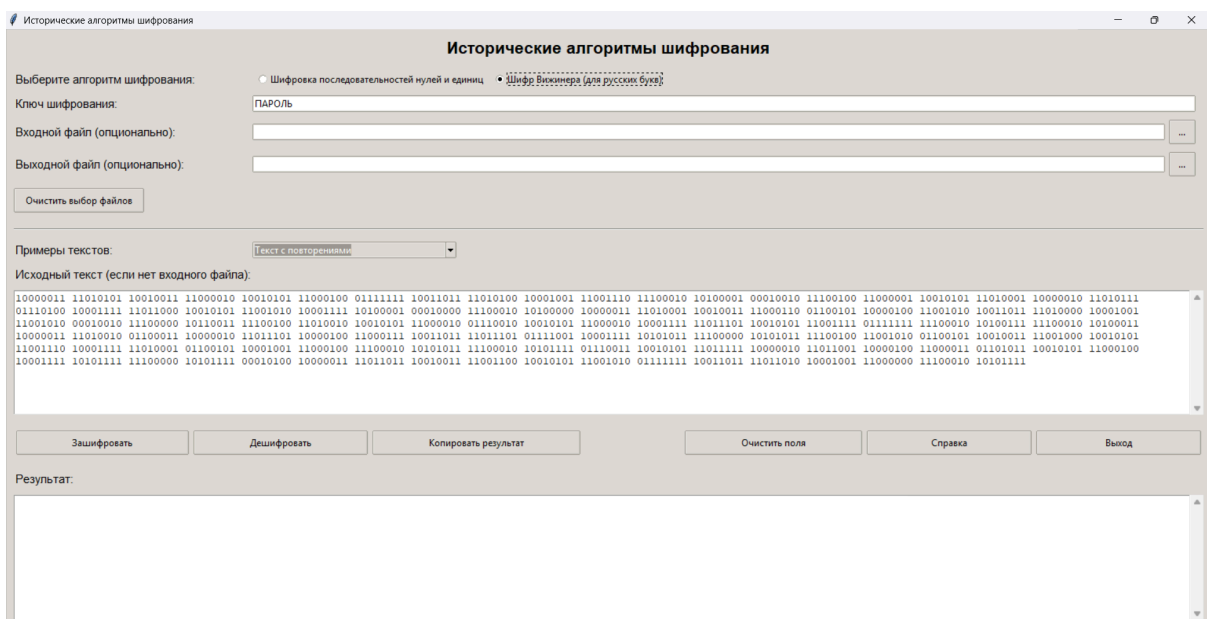


Рисунок 10 – Графический интерфейс для шифровки с помощью шифра Вижинера (для русских букв)

Примеры текстов: Короткий русский ▾

Исходный текст (если нет входного файла):

ПРИВЕТ МИР

Зашифровать Дешифровать Копировать результат

Результат:

ЯРЩРРО МЩЯ

Рисунок 11 – Шифровка с помощью шифра Вижинера (для русских букв).
Короткий русский текст

Примеры текстов: Короткий русский ▾

Исходный текст (если нет входного файла):

ЯРЩРРО МЩЯ

Зашифровать Дешифровать Копировать результат

Результат:

ПРИВЕТ МИР

Рисунок 12 – Дешифровка с помощью шифра Вижинера (для русских букв). Короткий русский текст

Примеры текстов: Длинный русский

Исходный текст (если нет входного файла):

Съешь же ещё этих мягких французских булок, да выпей чаю. Тридцать три богатыря, в чешуе златой, по очереди выходят из моря и дозором обходят дозором владения дядьки Черномора.

Результат:

БЪХЖЗ ЦЕ УЕВ ЭГЧБ ЪЯУЩ*С *БОЩПГЪВЩ*С БДЪЪЖ ДР НЧЯЕЪ ГЪН ГЯ*АЕАГК ОАИ ПЪЯПТЛЯК С ЗУДП* ШЪЛОЖИ КЪ КЧХЯРАШ ТИЕКУЯГ *Д МАЯК Ш *ЗУКАОЗ *ЪЕО*НЮ
УОШЪЪКЪ ТЪЛА*НЩИ АОДМЩ* ЖЕВЪЪЮОРР

Рисунок 13 – Шифровка с помощью шифра Вижинера (для русских букв).
Длинный русский текст

Примеры текстов: Длинный русский

Исходный текст (если нет входного файла):

БЪХЖЗ ЦЕ УЕВ ЭГЧБ ЪЯУЩ*С *БОЩПГЪВЩ*С БДЪЪЖ ДР НЧЯЕЪ ГЪН ГЯ*АЕАГК ОАИ ПЪЯПТЛЯК С ЗУДП* ШЪЛОЖИ КЪ КЧХЯРАШ ТИЕКУЯГ *Д МАЯК Ш *ЗУКАОЗ *ЪЕО*НЮ
УОШЪЪКЪ ТЪЛА*НЩИ АОДМЩ* ЖЕВЪЪЮОРР

Результат:

СЪЕШЬ ЖЕ ЕЩЁ ЭТИХ МЯГКИХ *ФРАНЦУЗСКИХ БУЛОК ДА ВЫПЕЙ ЧАЮ ТРИДЦАТЬ ТРИ БОГАТЫРЯ В ЧЕШУЕ ЗЛАТОЙ ПО ОЧЕРЕДИ ВЫХОДЯТ ИЗ МОРЯ И ДОЗОРОМ ОБХОДЯТ ДОЗОРОМ ВЛАДЕНИЯ ДЯДЬКИ ЧЕРНОМОРА

Рисунок 14 – Дешифровка с помощью шифра Вижинера (для русских букв). Длинный русский текст

Примеры текстов: Текст с повторениями

Исходный текст (если нет входного файла):

ААА БББ ВВВ ГГГ ДДД ЕЕЕ ЁЁЁ ЖЖЖ ЗЗЗ ИИИ ЙЙЙ ККК ЛЛЛ МММ ННН ООО ППП

Зашифровать Дешифровать Копировать результат

Результат:

ПАР МЭР ТРН ТГУ ПАУ ХУР ХЁЦ ТГЦ ЩЦУ ШИЩ ХЁЩ ЫЩЦ ЫЛЬ ШИЬ ЮЬЩ ЮОЯ ЫЛЯ

Рисунок 15 – Шифровка с помощью шифра Вижинера (для русских букв).
Текст с повторениями

Примеры текстов: Текст с повторениями

Исходный текст (если нет входного файла):

ПАР МЭР ТРН ТГУ ПАУ ХУР ХЁЦ ТГЦ ЩЦУ ШИЩ ХЁЩ ЫЩЦ ЫЛЬ ШИЬ ЮЬЩ ЮОЯ ЫЛЯ

Зашифровать Дешифровать Копировать результат

Результат:

ААА БББ ВВВ ГГГ ДДД ЕЕЕ ЁЁЁ ЖЖЖ ЗЗЗ ИИИ ЙЙЙ ККК ЛЛЛ МММ ННН ООО ППП

Рисунок 16 – Дешифровка с помощью шифра Вижинера (для русских букв). Текст с повторениями

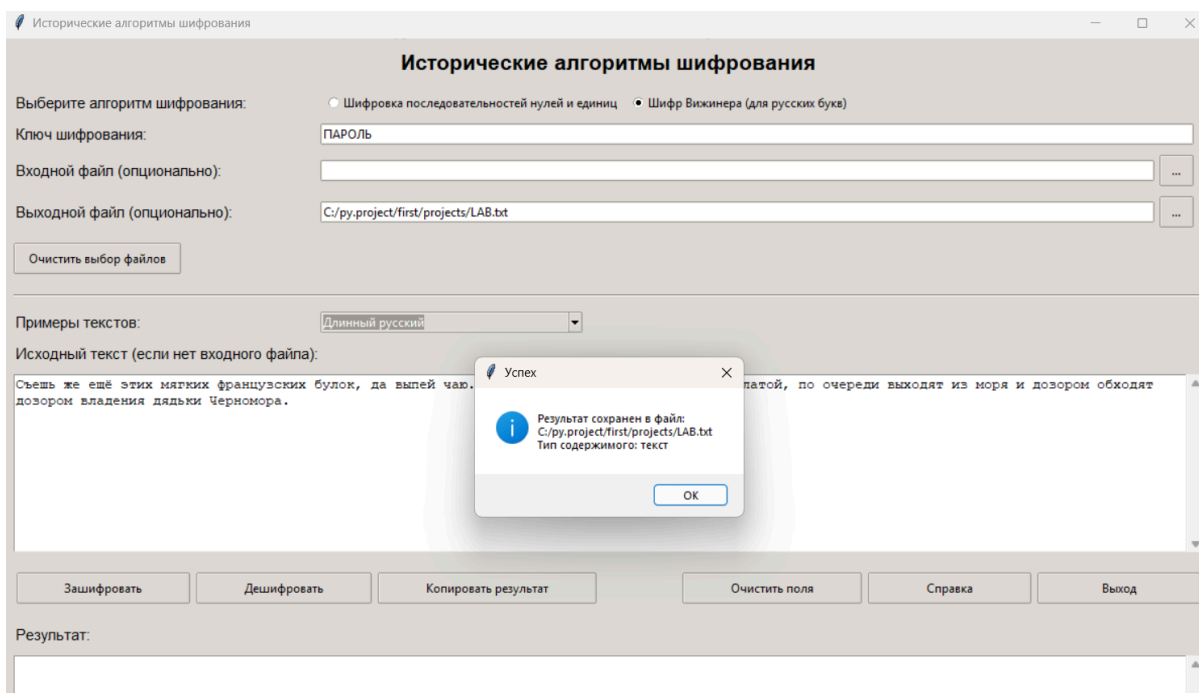


Рисунок 17 – Шифровка с помощью шифра Вижинера (для русских букв) с записью в файл

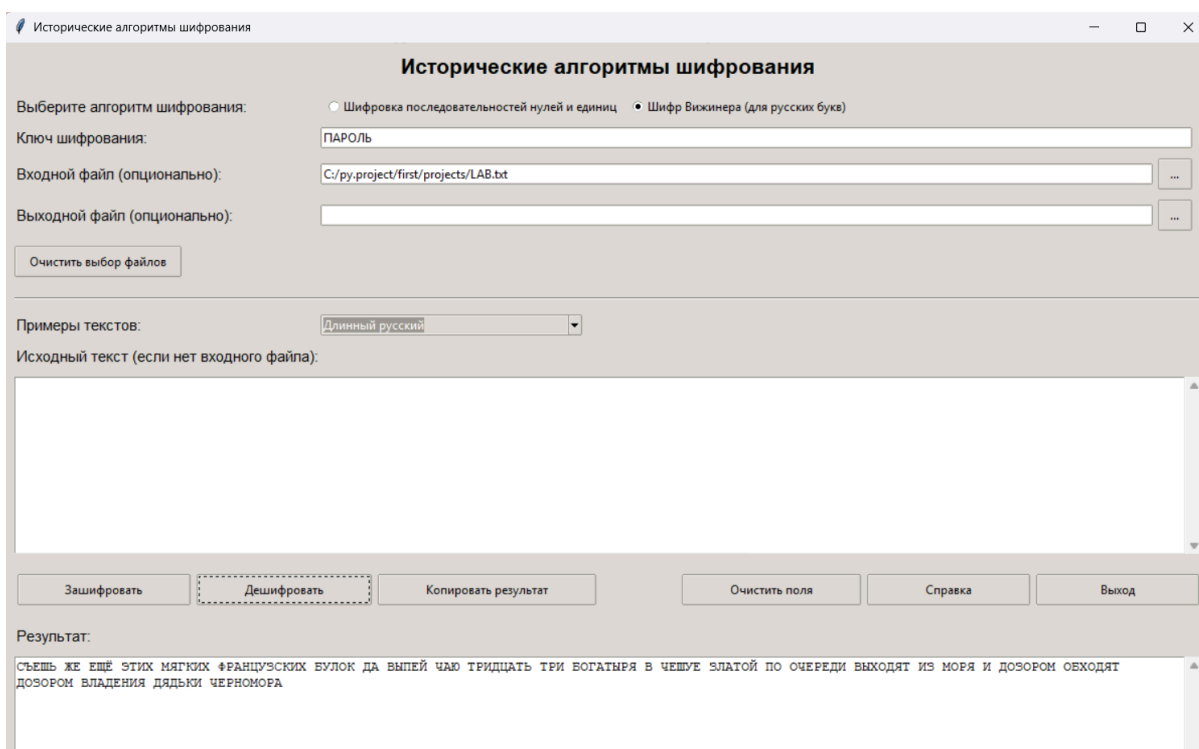


Рисунок 17 – Дешифровка с помощью шифра Вижинера (для русских букв) с чтением из файла

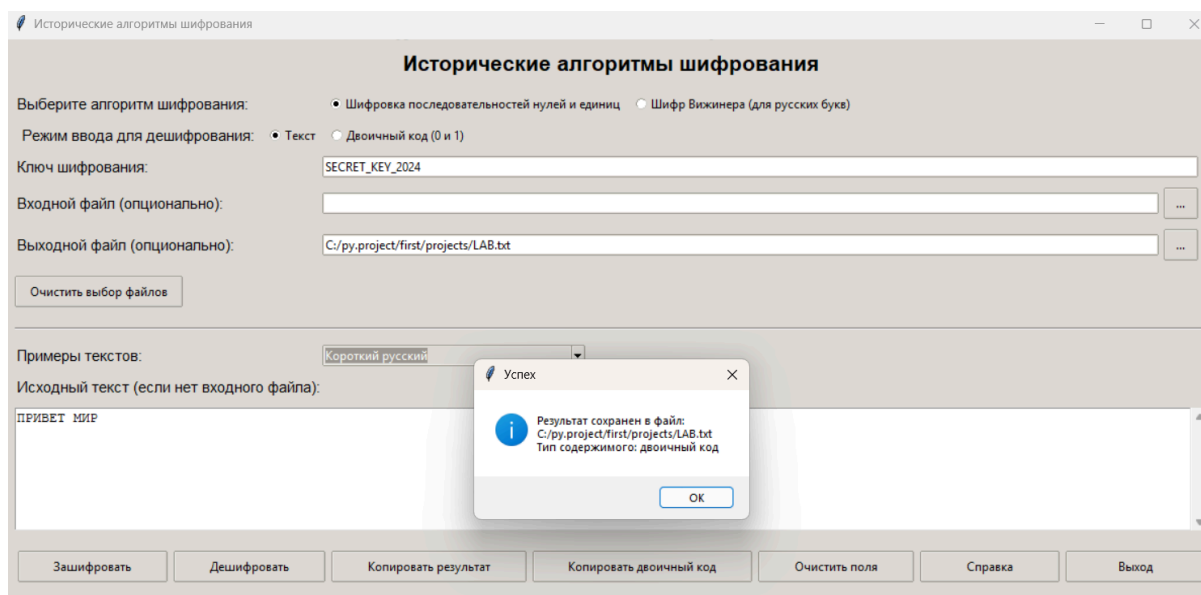


Рисунок 18 – Шифровка последовательностей нулей и единиц с записью в файл

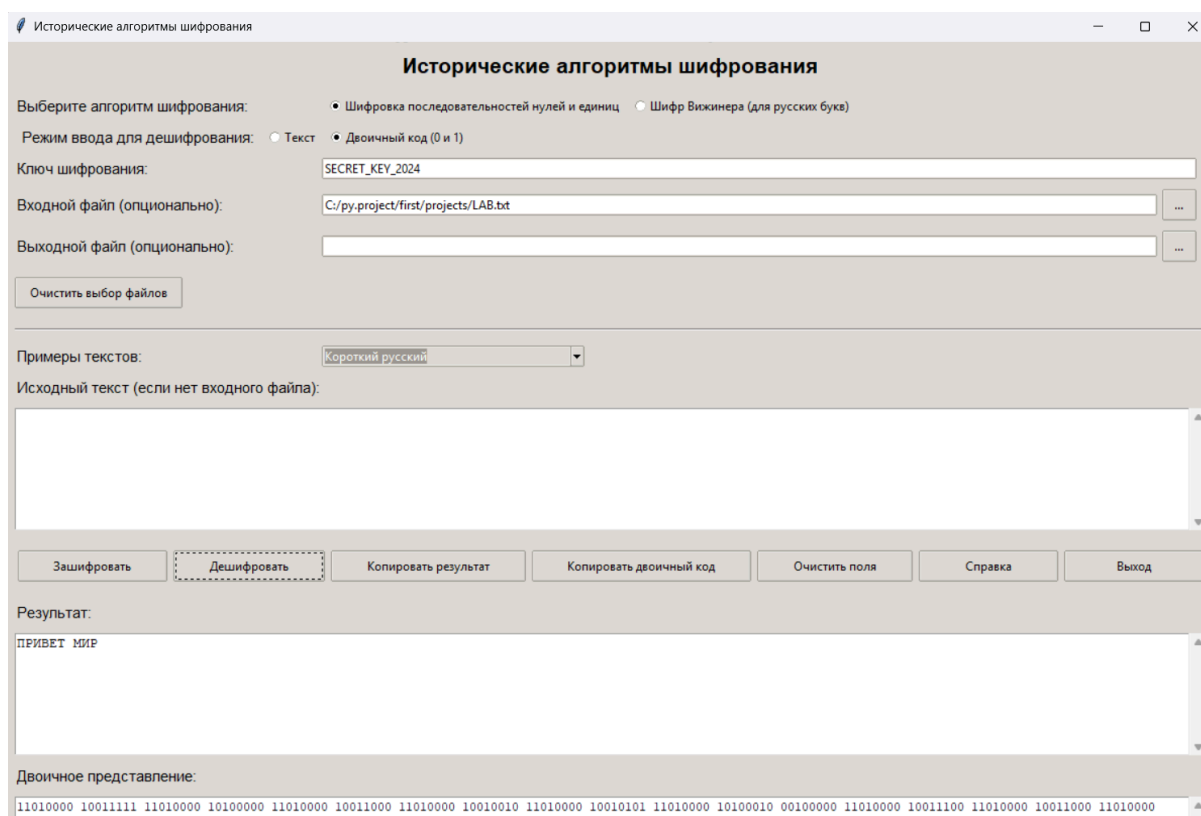


Рисунок 19 – Дешифровка последовательностей нулей и единиц с записью в файл

ЗАКЛЮЧЕНИЕ

В рамках лабораторной работы разработано функциональное приложение, демонстрирующее работу двух классических криптографических алгоритмов. Программа не только корректно реализует математические основы XOR-шифра и шифра Вижинера, но и предоставляет удобный, интуитивно понятный интерфейс для их исследования. Интеграция работы с файлами, система подсказок и горячих клавиш делает инструмент практичным для учебного процесса. Программа готова к использованию для наглядного изучения принципов криптографии.